



Categoría a participar: viii. Inteligencia Artificial (agentes de IA, modelos locales y automatización de código).

Guía de preparación y sustento arquitectónico para la presentación ante el jurado

Visión General y Enfoque de Demostración

Centinela 503 es una plataforma de respuesta comunitaria *offline-first* diseñada para coordinar emergencias sin depender de conectividad externa. Para nuestra presentación en el concurso ASPROC (CONINFO 2026), el proyecto operará como un piloto de campo enfocado en la versatilidad de un ecosistema de **propósito general**.

El despliegue utilizará **datos simulados y alertas aleatorias** detonadas mediante un botón físico conectado al nodo de campo. En lugar de simular sensores estáticos (como humo o temperatura), inyectamos cadenas de texto variadas. Esto prueba que nuestra Inteligencia Artificial local puede comprender semánticamente el problema, clasificarlo y asignar recursos, operando en múltiples contextos sin modificar el código fuente.

Núcleo Analítico mediante Inteligencia Artificial

Para cumplir de forma impecable con la categoría de Inteligencia Artificial del congreso, la plataforma emplea agentes de software y algoritmos ejecutados **100 % en local** dentro del backend, demostrando autonomía operativa y automatización de procesos sin requerir servicios externos.

Triaje Automatizado (Procesamiento de Lenguaje Natural)

Cuando ingresa una alerta de texto cruda (ej. "persona atrapada" o "inundación"), el motor de NLP analiza su semántica. Automáticamente determina la categoría del incidente y le asigna un nivel de **Prioridad Sugerida** (Alta, Media, Baja). Este agente de software actúa como un primer interviniente digital para acelerar la toma de decisiones.

Agrupamiento Espacial y Distancia Haversine

Para evitar la saturación visual y operativa, el sistema aplica algoritmos de *Machine Learning* no supervisados para agrupar reportes. Calcula las distancias geoespaciales exactas entre cada alerta ciudadana utilizando la **Fórmula del Semiverseno (Haversine)**, que toma en consideración la curvatura de la Tierra. Si múltiples incidentes ocurren a menos de **50 metros** y dentro de una ventana temporal de 15 minutos, el modelo los clusteriza en un único evento mayor.

Agente de Asignación y Sugerencia de Recursos

Con base en la categoría inferida por la IA y la ubicación exacta del cluster (Haversine), un algoritmo de emparejamiento cruza las habilidades requeridas (ej. rescate o medicina) con las habilidades registradas de cada brigada disponible. El agente genera una tabla de puntajes (*scoring*) y recomienda al Coordinador de Emergencia cuál es la unidad más óptima y cercana para ser despachada.

Alineación con los Objetivos de Desarrollo Sostenible (ODS)

El impacto y escalabilidad de Centinela 503 se fundamentan en tres ODS prioritarios:

ODS 11: Ciudades y Comunidades Sostenibles

La plataforma fortalece la resiliencia urbana. El algoritmo espacial (Haversine) evita la saturación informativa consolidando reportes concurrentes, garantizando una coordinación ordenada en crisis.

ODS 9: Industria, Innovación e Infraestructura

Integramos hardware de bajo costo con infraestructura 100 % local, eliminando la dependencia de servidores en la nube. Representa una infraestructura de telecomunicaciones tolerante a fallas.

ODS 16: Paz, Justicia e Instituciones Sólidas

El triaje automatizado de la IA reduce el tiempo de identificación de alertas críticas, apoyando a instituciones como Protección Civil. El Dashboard audita y transparenta las operaciones.

Stack Tecnológico (Componentes)

El proyecto se construyó integrando rigurosamente las siguientes tecnologías:

Software y Arquitectura:

- **Backend:** Python 3, framework **FastAPI** para la creación de la API REST, y base de datos relacional **SQLite** optimizada para concurrencia local.
- **Inteligencia Artificial:** Motor de **NLP** semántico y clusterización aplicando la **Fórmula de Haversine**.
- **Frontend:** Interfaz **Vanilla JS** (JavaScript puro), diseño fluido con **Tailwind CSS**, y motor de mapas interactivos renderizados *offline* con **Leaflet.js**.
- **Seguridad:** Sistema de sesión protegido mediante autenticación sin estado con Tokens **JWT**.

Hardware y Comunicaciones:

- **Cómputo:** Microcontroladores **ESP32** (núcleo dual de alta eficiencia).
- **Radiofrecuencia:** Módulos Transceptores LoRa **RFM95** operando a **915MHz**.
- **Antenas:** Antenas omnidireccionales de **6dBi** para maximizar la ganancia.
- **Energía:** Módulo de carga tipo **Power Bank (5V)** para autonomía en campo.

▶ Guía de Ejecución Paso a Paso

Para levantar el ecosistema el día de la defensa, sigue este procedimiento garantizando que no haya dependencia de internet:

Paso 1: Inicialización del Hardware y RF

Conectar el **Nodo Receptor (Base)** a un puerto USB de la computadora designada como servidor. Asegurarse de que el sistema reconozca el puerto serie (ej. `/dev/ttyUSB0` o `COM3`). Posteriormente, encender el Nodo Emisor de campo garantizando que tenga carga en la batería. Operamos LoRa con **Spreading Factor 10 (SF10)** para penetración de obstáculos.

Paso 2: Ejecución del Backend (API e Inteligencia Artificial)

Abrir una terminal, activar el entorno virtual y levantar el servidor de FastAPI. Desde la raíz del directorio backend, ejecutar el comando:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

El servidor cargará los modelos locales de IA en memoria y abrirá la escucha serial.

Paso 3: Arranque del Frontend (Dashboard Interactivo)

Levantar el cliente web. Puedes utilizar una extensión como Live Server en VS Code o un servidor estático rápido (ej. `python -m http.server`). Abre el navegador en la ruta provista (ej. `http://localhost:5500`) e inicia sesión en el panel para comenzar a visualizar y coordinar alertas mediante la IA.

? Simulador de Preguntas Complejas del Jurado (Killer Questions)

1. Su categoría es la VIII (Inteligencia Artificial). ¿Dónde se ejecuta la IA y por qué no usar la API de ChatGPT u OpenAI?

Sustento Técnico: Nuestro sistema es de respuesta ante desastres donde las telecomunicaciones fallan por completo. Dependiendo de la API de OpenAI significa requerir acceso a internet, lo cual es inaceptable en una catástrofe. Por ello, toda nuestra Inteligencia Artificial (procesamiento de lenguaje natural, clusterización Haversine y asignación de recursos) está desarrollada nativamente en Python y se ejecuta localmente en el procesador del servidor, sin latencia de red ni dependencia de terceros.

2. ¿Por qué simular datos en lugar de usar sensores de humo/humedad reales?

Sustento Técnico: Un sensor de humo nos encasilla como una simple "alarma de incendios". Al inyectar textos descriptivos variables desde el botón, demostramos que nuestro agente de software (NLP) lee, clasifica y prioriza emergencias de naturaleza completamente distinta (sismos, disturbios civiles, colapsos estructurales) de manera automática. Exhibimos un ecosistema de propósito general verdaderamente inteligente.

3. ¿Qué ocurre si múltiples alertas solicitan la misma brigada simultáneamente?

Sustento Técnico: Utilizamos transacciones atómicas y bloqueos de escritura seguros nativos de la base de datos (SQLite / FastAPI). La primera solicitud asegura la reserva de la brigada a nivel base de datos; cuando la segunda solicitud (milisegundos después) es evaluada, la transacción le informa de la falta de disponibilidad y el motor de recomendación de IA recalcula instantáneamente para sugerir a la segunda mejor brigada disponible.

4. ¿Cómo evitan colapsar la memoria del equipo usando Machine Learning local?

Sustento Técnico: Descartamos Redes Neuronales Profundas (LLMs de cientos de GB) que no son viables en equipos de gama media u operaciones de campaña. En su lugar, hemos implementado una IA basada en algoritmos estadísticos altamente eficientes. El análisis de densidad espacial y el cálculo trigonométrico de coordenadas consumen muy pocos ciclos de CPU, permitiendo correr nuestro modelo con alto rendimiento y nula exigencia de tarjetas de video (GPU).

5. ¿Por qué usar módulos LoRa y crear su propia red en lugar de hacer una app conectada a Tigo o Claro?

Sustento Técnico: El problema de depender de redes comerciales es el **Punto Único de Fallo** (*Single Point of Failure*). En desastres como los terremotos de 2001, las redes comerciales colapsan inmediatamente por saturación, cortes de fibra óptica o caída de energía en las antenas LTE. Nuestra red LoRa es descentralizada, independiente de los Proveedores de Servicios de Internet (ISP) y funciona con baterías, garantizando comunicación vital cuando todo lo demás falla.

6. ¿Qué pasa si el servidor central se apaga, se reinicia accidentalmente o el cable USB se desconecta?

Sustento Técnico (Arquitectura de Resiliencia): Si el servidor cae, el frontend web detecta la caída de la API y muestra una alerta de "Sin Conexión", pero gracias a los Tokens JWT, la sesión no se pierde. En el backend, usamos FastAPI con una base de datos SQLite persistente. Al reiniciar la PC y encender el script, el sistema recupera su estado anterior exacto y vuelve a abrir el puerto serial para recibir las nuevas alertas en segundos.

7. Este prototipo cubre el campus (2km). ¿Cómo escalarían este proyecto para cubrir todo San Miguel?

Sustento Técnico: La red LoRa permite topologías de **Malla (Mesh)** o el uso de repetidores. Colocando antenas LoRa en puntos de alta elevación (como en la cima del Volcán Chaparrastique o en antenas de radio existentes en el Cerro de Las Figuras), los nodos emisores triangularían la señal, rebotando el mensaje de nodo en nodo hasta llegar al Centro de Operaciones, cubriendo decenas de kilómetros sin costo adicional de transmisión.

8. Si ganan hoy, ¿en qué zonas de mayor riesgo de San Miguel implementarían este proyecto mañana mismo y por qué?

Sustento Técnico (Protección Civil y Gestión de Riesgos): Desplegaríamos la topología inicial dividida en dos ejes críticos de vulnerabilidad histórica en San Miguel:

1. Eje de Inundaciones (Río Grande de San Miguel): Instalaríamos Nodos Emisores en **Cantón La Canoa, Caserío Santa Fidelia y Colonia Carrillo**. Son planicies de inundación que rutinariamente quedan incomunicadas y aisladas durante tormentas severas. Un nodo allí serviría como alerta temprana del desbordamiento, transmitiendo por LoRa mucho antes de que las líneas telefónicas comerciales colapsen por la lluvia.

2. Eje Volcánico y Lahares (Volcán Chaparrastique): Ubicaríamos emisores en **Las Placitas, Caserío La Cruz y El Conacastal**. Estas zonas altas sufren de un severo efecto de “sombra celular” (cero cobertura LTE) y están expuestas a flujos de escombros (lahares) y enjambres sísmicos. La robustez física y autonomía de nuestra placa ESP32+RFM95 garantiza comunicación desde la falda del volcán hacia el valle.

Arquitectura de Despliegue: Mientras estos Nodos Emisores quedan en las zonas de riesgo, el **Nodo Receptor y el Servidor Local (Dashboard + Inteligencia Artificial)** se instalarían estratégicamente en el **Centro de Operaciones de Emergencia (COE) municipal** o en la delegación urbana de Protección Civil, centralizando la toma de decisiones.

Si el jurado pregunta: “¿Realizaron un estudio de campo previo para afirmar esto?”

Respuesta: “Sí. Realizamos un cruce preliminar de información utilizando las matrices de vulnerabilidad histórica de Protección Civil y los mapas de riesgo del MARN (Ministerio de Medio Ambiente). Nuestro estudio determinó que la falla crítica de San Miguel no es la falta de planes de contingencia, sino el **colapso comunicacional** (apagones y pérdida de señal celular) durante las primeras horas de una catástrofe (Golden Hours). Por ello, el hardware de Centinela 503 está diseñado para mitigar exactamente este punto de quiebre.”